TED UNIVERSITY

Faculty of Engineering

CMPE 491/SENG491

Senior Project I

LLM Valley

Project Specifications Report

TEAM 21

14.11.25

by

Yusuf Sadi Pesen

Deniz Şeker

Egemen Asım Ersoy

Reyhan Sena Çimen

Supervisor:

Dr.Emin Kuğu

Jury Members:

Dr. Ulaş Güleç

Dr. Eren Ulu

1. Introduction 2
1.1 Description 2
1.2 Constraints 2
1.3 Professional and Ethical Issues 4
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 Assumptions and Dependencies 6
3. External Interface Requirements 6
3.1 User Interfaces 6
3.2 Hardware Interfaces 7
3.3 Software Interfaces 7
3.4 Communications Interfaces 7
4. System Features 7
4.1 System Feature: Conversation System 8
4.2 System Feature: Farming System 8
4.3 System Feature: Trading System 9
5. Requirements 10
5.1 Functional Requirements 10
5. 2 Non-Functional Requirements 10
5.3 Development Constraints 11
6. References 11

1. Introduction

1.1 Description

LLM Valley is a narrative life simulation demo that uses Large Language Model (LLM) technology to drive dynamic NPC (non-player character) conversations. Players can farm, trade, and build relationships with 4–5 NPCs in a small cozy village, entirely through natural keyboard-based dialogue.

The project focuses on showcasing the integration of AI text generation in real-time gameplay. NPCs respond dynamically based on player input and simple keyword-based stat updates rather than memory-heavy systems. The demo emphasizes clarity, emotional depth, and technical feasibility within a short 3-month development cycle.

The core gameplay loop includes:

- **Farming:** Watering, planting, and harvesting 3 simple crops
- **Talking:** Conversing with NPCs via keyboard input
- **Trading:** Buying seeds and gifts, selling fresh produce
- **Sleeping:** Resting to restore mental energy and advance time

The goal is to create a 20–30 minute demo that demonstrates believable AI-driven interactions while keeping systems minimal and efficient.

The website of the project can be accessed by https://llmvalley.framer.website/

1.2 Constraints

Economic Constraints

- **LLM API Costs:**
  The project's API calls to the Deepseek API or Gemini API which are free.

Environmental Constraints

- The game runs on mid-range PCs (2018+), requiring no additional hardware or manufacturing.
- No physical materials are consumed during production, resulting in a negligible environmental footprint.

Social Constraints

- The game aims for inclusivity and positive communication.
- NPC dialogue is designed to avoid offensive or discriminatory responses through careful prompt engineering and keyword filtering.

Political Constraints

- The content avoids political subjects, affiliations, or ideological messaging.
- NPCs' dialogues focus on personal and community themes, not political discourse.

Ethical Constraints

- LLM-generated text must follow appropriate behavior guidelines to prevent biased, offensive, or harmful output.
- The development team follows the ACM Code of Ethics and IEEE Software Engineering Code of Ethic**s**, ensuring fairness, privacy, and user respect.

Health and Safety Constraints

- The game involves no physical hazards and minimal cognitive load.
- Color palettes and text interfaces are selected for visibility and accessibility.
- No rapid flashing visuals or stressful gameplay elements are present.

Manufacturability Constraints

- The project produces a digital software demo only, there are no physical manufacturing needs.
- All systems are modular to ensure maintainability and upgradability.

Sustainability Constraints

- Minimal computational requirements ensure the demo runs efficiently on existing hardware.
- The use of open assets and scalable code architecture promotes future development sustainability.

1.3 Professional and Ethical Issues

The LLM Valley project integrates AI in player interaction, which introduces ethical and professional responsibilities related to transparency, reliability, and data handling.

Ethical Principles Applied:

- **Transparency:** The game will clearly inform users that NPCs' dialogues are AI-generated.
- **Data Privacy:** No player input is stored or analyzed beyond session context, no personal data is collected.
- **Safety and Respect:** LLM prompts will include filters to avoid generating harmful, discriminatory, or explicit content.
- **Integrity:** Developers will report any issues related to LLM misuse or output bias during testing.
- **Accountability:** The project will adhere to IEEE and ACM codes by ensuring honesty, quality, and responsibility in AI usage.

2. Overall Description

2.1 Product Perspective

LLM Valley is a standalone demo game designed to demonstrate real-time integration of LLMs in interactive NPC dialogue systems. It is not part of an existing product family and is developed as an independent project. The system uses a simple architecture that connects player text input to an LLM API, processes responses, and updates gameplay variables accordingly.

The software consists of the following main components:

- **Core Game System:** Handles movement, time progression, farming, and trading.
- **Dialogue System:** Manages user text input, sends it to the LLM API, and displays generated NPC responses.
- **Stat System:** Tracks relationship and mental attributes (Love, Trust, Friendship, Mental, Popularity).
- **Data Handling Layer:** Manages temporary in-session data without permanent storage.

2.2 Product Functions

The main functions of the software are:

- Allow the player to move freely in a 2D top-down environment.
- Enable farming actions such as planting, watering, and harvesting crops.
- Provide trading functionality with NPCs to buy and sell items.
- Facilitate natural conversation with NPCs using free-text input.
- Track and update player statistics through keyword-based analysis of dialogue.
- Let the player rest/sleep to advance in-game days and refresh mental energy.

- Save and load basic gameplay progress (day count, crops, stats).

2.3 User Classes and Characteristics

- **General Player:** Has basic knowledge of PC games and uses keyboard and mouse. Interested in narrative and AI-driven experiences.
- **Developer / Evaluator:** Focuses on testing system performance, LLM reliability, and prompt engineering outcomes. Has basic technical background in software systems.

Both user classes expect intuitive controls, short response times, and stable performance.

2.4 Operating Environment

- **Hardware:** Runs on standard Windows PCs (2018 or newer).
- **Software:** Windows 10+ OS, game built using Unity Engine.
- **Network:** Internet connection required for API calls to LLM service.
- **Display:** Fixed resolution 1280×720, keyboard and mouse input only.

2.5 Design and Implementation Constraints

- No 3D assets or animations, visuals remain static 2D.
- Only one playable map and limited NPC count.
- Use of free or open-source assets.
- Integration restricted to Deepseek or Gemini API.
- Three-month development deadline divided into clear phases.

2.6 Assumptions and Dependencies

- LLM APIs (Deepseek or Gemini) remain accessible and stable.
- Players have reliable internet connections.
- Unity engine supports necessary text and UI features.
- No additional external database or backend system is used.

3. External Interface Requirements

3.1 User Interfaces

- **Main Game Screen:** Displays the player's farm, NPCs, and environment in a top-down 2D layout.
- **Dialogue Window:** Text input box for player messages and dialogue display area for NPC responses.
- **Stat Display:** On-screen bars showing current Love, Trust, Friendship, Mental, and Popularity values.
- **Shop Screen:** Simple trading menu for buying and selling crops or gifts.
- **Sleep Menu:** Option to rest and advance to the next day.

Interface style is minimal and readable with soft colors, consistent font usage, and accessibility-focused contrast.

3.2 Hardware Interfaces

- Keyboard and mouse are the only input devices.
- No controller or touch support.
- Internet connection is required for communication with LLM API.

3.3 Software Interfaces

- **Unity Engine:** Game engine used for main logic, UI, and 2D rendering.
- **Deepseek or Gemini API:** Provides LLM-based text generation for NPC responses.
- **Local JSON Data:** Stores temporary gameplay data (e.g., crops, stats, current day).

3.4 Communications Interfaces

- Uses HTTPS for secure API communication with LLM services.
- Simple text-based requests and responses (JSON format).

- No persistent user data or external authentication system.

4. System Features

4.1 System Feature: Conversation System

ID and Name: SF-1 – NPC Dialogue System
Actors: Player, LLM API
Description: Enables real-time text conversation between the player and NPCs, generating emotionally appropriate and contextually relevant responses.
Trigger: Player inputs a message into the text box.
Preconditions: Player must be near an NPC and have available daily conversation slots.
Postconditions: NPC responds within 3–5 seconds; relationship stats may update.
Normal Flow:

1. Player types message.
2. Message sent to LLM API.
3. API response displayed as NPC dialogue.
4. Relevant stats adjusted based on keywords.

Alternative Flows:

- If API response fails, display a fallback message (“Sorry, I didn’t catch that.”).

Exceptions:

- Lost internet connection → prompt player to retry.

Non-Functional Requirements:

- Response time ≤ 5 seconds in 90% of cases.
- NPC responses filtered for safety and appropriateness.

Functional Requirements:

- REQ-1: Game must send player input text to LLM API.
- REQ-2: System must display the generated response in UI.
- REQ-3: Stat updates occur based on predefined keywords.

4.2 System Feature: Farming System

ID and Name: SF-2 – Crop Interaction
Actors: Player
Description: Allows the player to plant, water, and harvest crops in a small farm plot.
Trigger: Player selects a farming action near a plot.
Preconditions: Player has seeds in inventory.
Postconditions: Crop status updated (growth stage advanced).
Normal Flow:

1. Player plants a seed.
2. Player waters it daily.
3. Crop matures and becomes harvestable.

Alternative Flows:

- If neglected for too long, crop withers.

Exceptions:

- Inventory full; cannot collect harvested item.

Functional Requirements:

- REQ-4: Game must allow planting of Tomato, Carrot, and Wheat.
- REQ-5: Growth state updates per in-game day.

4.3 System Feature: Trading System

ID and Name: SF-3 – Shop and Trade
Actors: Player, Shop NPC
Description: Allows buying and selling of seeds, crops, and gifts at fixed prices.
Trigger: Player interacts with the shop NPC.
Preconditions: Player must have enough in-game currency.
Postconditions: Player inventory and balance update accordingly.
Normal Flow:

1. Player opens shop menu.
2. Selects item to buy or sell.
3. Transaction updates inventory and money.

Alternative Flows:

- Insufficient balance message displayed.

Functional Requirements:

- REQ-6: Shop interface displays available items and prices.
- REQ-7: Transaction success/failure messages shown.

5. Requirements

5.1 Functional Requirements

- **Core Gameplay Systems**
  - 2D top-down movement system
  - Farming system with crops
  - Trading interface (buy/sell, fixed prices)
  - Day/Night and sleep cycle
  - Stat tracking (Love, Trust, Friendship, Mental, Popularity)
- **LLM Integration**
  - Integration with Deepseek API or Gemini API
  - Keyword-based stat updates
  - NPCs respond within 3–5 seconds
- **UI and Interaction**
  - Text input box for free-form conversation
  - Stat bars for player attributes
  - Simple 2D farm UI
  - One-shop trading screen
- **Progression System**
  - Unlock access to new NPCs as Popularity increases
  - Simple fetch quests
  - Day progression through sleep
- **Save/Load System**
  - Single save slot that records day count, stats, and crops

5. 2 Non-Functional Requirements

- **Performance:**
  LLM response time ≤ 5 seconds in 90% of cases
- **Stability:**
  Player can complete 7 in-game days without crashes
- **Usability:**
  Players can understand controls and dialogue system without tutorial
- **Compatibility:**
  Runs smoothly on Windows PC (mid-range, 2018+ hardware)
- **Security:**
  No user data storage, only temporary text context is sent to LLM API
- **Maintainability:**
  Modular structure to allow replacement of LLM API or expansion of content

5.3 Development Constraints

- No sound or animations (static visuals acceptable)
- Only one map/scene
- No controller or multiplayer support
- Fixed resolution, keyboard + mouse only
- Strict 3-month timeline divided into 3 phases:
  - **Month 1:** Core systems (movement, UI, farming, first NPC)
  - **Month 2:** Stats, trading, additional NPCs
  - **Month 3:** Testing, polish, optimization

---

6. References

[1] ACM Code of Ethics and Professional Conduct
[2] Software Engineering Code of Ethics, IEEE Computer Society
[3] IEEE Code of Ethics
[4] *Computer and Information Ethics*, Stanford Encyclopedia of Philosophy
[5] *LLM Valley Game Design Document, Version 1.0*